

Feed Forward Neural Network

Machine Learning — Core Concepts & Engineering Guide

Subject: Machine Learning

Topic: Neural Networks

Level: Engineering UG/PG

1. Introduction

A **Feed Forward Neural Network (FFNN)** — also called a Multi-Layer Perceptron (MLP) — is the simplest class of artificial neural network. Information travels in one direction only: from the **input layer** through one or more **hidden layers** to the **output layer**. There are no cycles or feedback loops, which makes FFNNs mathematically tractable and computationally efficient.

FFNNs are the building block of modern deep learning. They are used in image classification, speech recognition, medical diagnosis, financial forecasting, and many other domains where a mapping from inputs to outputs must be learned from data.

2. Architecture

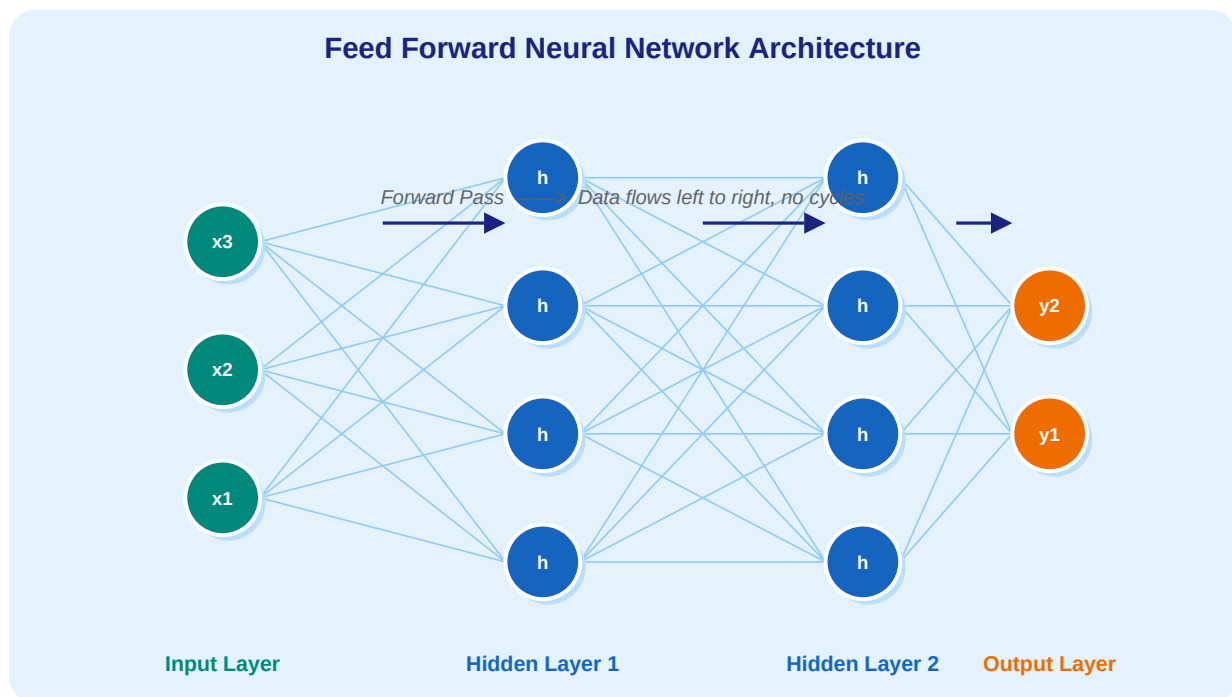


Figure 1 — FFNN with 3 inputs, 2 hidden layers (4 neurons each), and 2 outputs

Layer	Role	Typical Activation
Input Layer	Receives raw features ($x_1, x_2, \dots, x_n$)	None (identity)
Hidden Layer(s)	Learns non-linear representations of data	ReLU, Tanh, Sigmoid

Output Layer	Produces final prediction or class probability	Softmax / Linear / Sigmoid
--------------	--	----------------------------

3. Mathematical Foundation

3.1 Forward Propagation

For each layer l , the pre-activation z and post-activation a are computed as:

$$z[l] = W[l] \cdot a[l-1] + b[l] \quad a[l] = f(z[l])$$

where $W[l]$ is the weight matrix, $b[l]$ is the bias vector, and f is the activation function.

3.2 Loss Function

Task	Loss Function	Formula
Binary Classification	Binary Cross-Entropy	$L = -[y \log(\hat{y}) + (1-y)\log(1-\hat{y})]$
Multi-class	Categorical Cross-Entropy	$L = -\sum (y_i \cdot \log(\hat{y}_i))$
Regression	Mean Squared Error	$L = (1/n) \sum (y_i - \hat{y}_i)^2$

3.3 Backpropagation (Learning)

Weights are updated using **Gradient Descent**:

$$W[l] = W[l] - \alpha \cdot \partial L / \partial W[l] \quad b[l] = b[l] - \alpha \cdot \partial L / \partial b[l] \quad \alpha = \text{learning rate (typically } 0.001 - 0.1)$$

4. Activation Functions

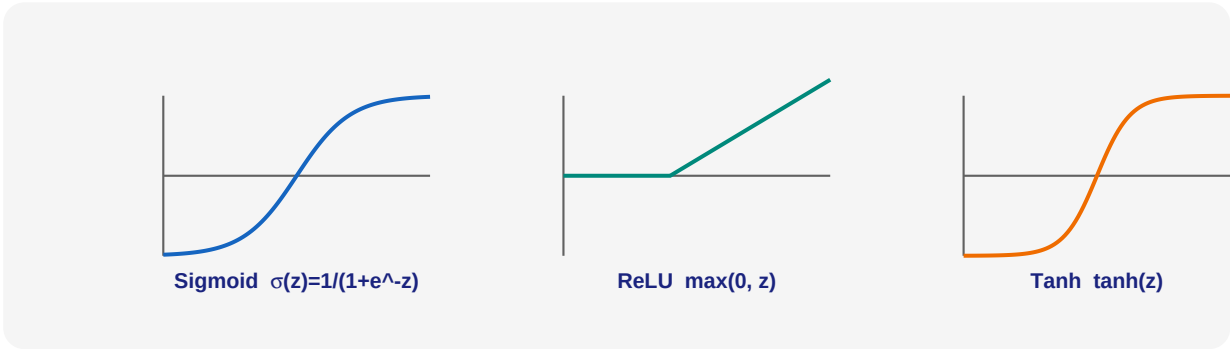


Figure 2 — Common activation functions used in hidden layers

Function	Formula	Range	Best Used For
Sigmoid	$1 / (1 + e^{-z})$	$(0, 1)$	Binary output layer
Tanh	$\tanh(z)$	$(-1, 1)$	Hidden layers (zero-centered)
ReLU	$\max(0, z)$	$[0, \infty)$	Deep networks (default choice)
Leaky ReLU	$\max(0.01z, z)$	$(-\infty, \infty)$	Avoids dying ReLU problem
Softmax	$e^{z_i} / \sum (e^{z_j})$	$(0, 1)$	Multi-class output layer

5. Worked Example — XOR Problem

XOR cannot be solved by a single perceptron (it is not linearly separable). A 2-layer FFNN solves it perfectly.

Network Design

- **Input layer:** 2 neurons (x1, x2)
- **Hidden layer:** 2 neurons (activation: Sigmoid)
- **Output layer:** 1 neuron (activation: Sigmoid)

Truth Table

x1	x2	XOR Output (y)
0	0	0
0	1	1
1	0	1
1	1	0

Python Implementation (Keras)

```
import numpy as np from tensorflow import keras X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]]) model = keras.Sequential([ keras.layers.Dense(2,
activation="sigmoid", input_shape=(2,)), keras.layers.Dense(1, activation="sigmoid") ])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(X, y, epochs=5000, verbose=0) print(model.predict(X).round())
```

After ~5 000 epochs the model predicts `[[0],[1],[1],[0]]` — a perfect XOR solution.

6. Key Hyperparameters

Hyperparameter	Typical Range	Effect
Learning Rate α	0.0001 – 0.1	Controls step size of weight update
Hidden Layers	1 – 5 (standard)	Depth of representation
Neurons per Layer	16 – 512	Width of each representation
Batch Size	16, 32, 64, 128	Stability vs. speed of training
Epochs	10 – 500+	Number of full passes over data
Dropout Rate	0.2 – 0.5	Regularization to prevent overfitting
Optimizer	Adam, SGD, RMSProp	Algorithm for gradient descent

7. Advantages & Limitations

Advantages	Limitations
------------	-------------

- Universal function approximator
- Handles high-dimensional data
- Flexible architecture design
- Effective with large datasets
- Works for regression & classification

- Requires large labeled datasets
- Computationally intensive to train
- Prone to overfitting (needs regularization)
- Black-box — low interpretability
- Vanishing gradient in deep networks

8. Summary

A Feed Forward Neural Network learns a mapping $y = f(x)$ through layers of weighted connections and non-linear activations. Training uses forward propagation to compute predictions, a loss function to measure error, and backpropagation with gradient descent to update weights. FFNNs are foundational to modern AI and serve as the basis for CNNs, RNNs, Transformers, and other advanced architectures.

References

- [1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [2] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- [3] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521, 436–444.
- [4] Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.